

AI Agentic Coding 방법론

Claude Code로 배우는 AI 협업 개발

3시간 특강 · 실습 중심 (GitHub Desktop · VS Code · Claude Code)

강남대학교 인공지능전공 · 인공지능서비스개발II

목차

PART 0	들어가며	
PART 1	환경 준비 — GitHub Desktop으로 시작하기	
PART 2	Claude Code 제대로 이해하기	
PART 3	프로젝트 컨텍스트 관리 — CLAUDE.md · AGENTS.md	
PART 4	명세로 시작하기 — PRD·TRD·REQUIREMENTS.md, SDD	
PART 5	AI와 협업하는 개발 방법론	
PART 6	종합 실습 — SDD + TDD 결합 미니 프로젝트	
PART 7	마무리	

00

PART 0

들어가며

- 인사 및 목표 안내
- "바이브 코딩"의 함정 라이브 데모
- 오늘 하루의 흐름 미리보기

오늘 배우고 나면

- Claude와 대화는 해보셨죠? 오늘은 그 Claude가 여러분 컴퓨터의 실제 파일을 직접 열고, 고치고, 실행까지 하는 모습을 봅니다.
- 이 수업 후, 여러분이 직접 AI에게 웹페이지 기능 하나를 통째로 맡겨서 완성시킬 수 있습니다.
- **오늘의 결과물**
 - 각자 미니 웹페이지(할 일 관리 앱)에 새 기능 하나를 실제로 추가해서 가져가기

"바이브 코딩"의 함정

- 애매한 지시를 한다면?
 - 예: "이 페이지 좀 더 좋게 만들어줘"
- 예상되는 문제
 - 불필요한 파일 변경
 - 기존 기능이 깨짐
 - 테스트 없이 "다 했다"고 보고하지만 실제로는 요구사항을 오해

"사람 후배한테 '알아서 잘 해줘'라고 시켰을 때랑 똑같죠? AI도 마찬가지입니다."

오늘 하루의 흐름

1

저장소 받아오기

GitHub Desktop

2

Claude Code 이해

도구

3

컨텍스트 관리

CLAUDE.md

4

명세로 못박기

REQUIREMENTS.md

5

협업 방법론

쪼개기·리뷰

6

실제로 완성하기

SDD+TDD

하루 종일 같은 실습용 저장소(mini-todo-web)를 계속 이어서 씁니다.

PART 1

환경 준비: GitHub Desktop으로 시작하기

- Git·GitHub·GitHub Desktop이 뭔가요?
- [실습] 저장소 클론하기
- [실습] Changes 탭에서 diff 보고 커밋하기

Git · GitHub · GitHub Desktop이 뭔가요?

Git

코드가 바뀔 때마다 "저장 지점"을
남기는 버전 관리 시스템

게임의 세이브 파일

GitHub

그 기록을 온라인에 올려 공유하고
함께 작업하는 서비스

코드의 온라인 저장소

GitHub Desktop

터미널 명령 없이 클릭만으로
Git·GitHub을 쓰게 해주는 프로그램

이것을 주로 사용

오늘 쓸 화면 3가지: Clone(내려받기) · Changes(무엇이 바뀌었는지) · Commit(저장 지점 남기기)

저장소 클론하기

- GitHub Desktop 실행
- File → Clone Repository 클릭
- URL 탭 선택 → <https://github.com/junghoege/mini-todo-web>
- Local Path(저장 위치) 확인 후 Clone 클릭
- "Open in Visual Studio Code" 버튼 클릭 → VS Code가 자동으로 열림

확인: VS Code 탐색기에 index.html · style.css · todoLogic.js · app.js가 보이면 성공

Changes 탭에서 diff 보고 커밋하기

- **GitHub Desktop 화면 구성**

- Changes 탭 — 지금까지 무엇이 바뀌었는지 줄 단위로 비교
- History 탭 — 과거 커밋 목록
- 커밋 메시지 입력창 + Commit to main 버튼

- **실습**

- index.html의 제목을 한 줄 고치기
→ Changes 탭에서 빨강/초록 비교 확인
→ 커밋 메시지 작성 후 Commit to main

오늘 하루, AI가 파일을 고칠 때마다 이 Changes 탭으로 돌아와 "무엇이 바뀌었는지" 확인하는 습관을 들입니다.

PART 2

Claude Code 제대로 이해하기

- Claude(챗봇) vs Claude Code(에이전트)
- [실습] VS Code 확장 설치 및 로그인
- [실습] 슬래시 커맨드 투어
- Skill과 Hook(선택) 자세히 알아보기

Claude(챗봇) vs Claude Code(에이전트)

Claude

대화만 합니다
물어보면 텍스트로 답할 뿐,
여러분 컴퓨터의 파일에는
손을 대지 못합니다

Claude Code

지금 열려 있는 프로젝트의
파일을 직접 읽고, 고치고,
필요하면 실행까지 해서
결과를 확인하는 "에이전트"

동작 원리: 계획(Plan) → 실행(파일 읽기/쓰기, 명령 실행) → 관찰(결과 확인) → 다시 계획 ... 반복

비유: 자동완성은 옆에서 다음 단어를 알려주는 사람, Claude Code는 "이 작업 좀 해줘"라고 맡기면 알아서 파일을 열어보고 고치고 실행해보고 확인까지 하는 신입 개발자

Claude Code for VS Code 설치 및 로그인

- VS Code에서 (Cmd/Ctrl+Shift+X 로) 확장(Extensions) 보기 열기
- 검색창에 "Claude Code" 입력 → Install 클릭
- 프로젝트 파일을 하나 열어두기 (파일이 열려 있어야 아이콘이 보임)
- 에디터 우측 상단의 스파크(*) 아이콘 클릭
- Sign in 클릭 → 브라우저 인증 → 이미 갖고 있는 Claude 계정으로 로그인
- 패널에 대화창이 뜨고 하단에 권한 모드가 보이면 성공

오늘은 이게 전부입니다. 터미널에 따로 설치할 필요가 없어요 — VS Code 확장 하나면 충분합니다.

슬래시 커맨드 투어

<code>/clear</code>	대화 맥락을 완전히 비우고 새로 시작
<code>/compact</code>	길어진 대화를 요약해서 컨텍스트 절약
<code>/model</code>	사용할 모델 전환
<code>/context</code>	지금 대화에 무엇이 로드돼 있는지 확인
<code>/help</code>	전체 커맨드 메뉴 훑어보기

`/context`는 3부에서 CLAUDE.md가 실제로 로드됐는지 확인하는 용도로 다시 씁니다.

Skill이란?

- 반복해서 채팅창에 붙여넣던 지침을 파일 하나(SKILL.md)로 저장해두고, 필요할 때 Claude가 알아서 불러 쓰거나 /스킬이름으로 직접 호출하는 기능
- 구조
 - 맨 위 description(언제 이 skill을 써야 하는지) + 아래 실제 지침(마크다운)
- 저장 위치
 - 개인용 ~/.claude/skills/ , 프로젝트용 .claude/skills/
 - 예: "PR 만들 때 체크리스트" skill을 한 번 만들어두면 이후엔 /pr-checklist 라고만 치면 됨

"Skill은 '내가 자주 시키는 방식'을 AI에게 한 번만 가르쳐두는 도구" — 오늘은 개념만, 다음 강의에서 직접 만들어봅니다.

보통 PR(풀 리퀘스트)을 만들 때마다 확인하는 절차가 있습니다.

"테스트 통과했는지, 디버그용 console.log가 남아있진 않은지, diff를 끝까지 읽어봤는지, 커밋 메시지가 명확한지" 같은 것들이요. 이걸 Claude Code한테 매번 시키려면, 채팅창에 그 체크리스트 전문을 매번 복사해서 붙여넣어야 합니다. 귀찮고, 사람마다 조금씩 다르게 적을 수도 있습니다.

Skill은 이 체크리스트를 파일로 딱 한 번 저장해두는 겁니다.

예를 들면 .claude/skills/pr-checklist/SKILL.md 라는 파일을 이렇게 만듭니다.

```
---
description: PR을 만들기 전에 체크리스트를 확인한다
---

## PR 체크리스트
1. 테스트를 모두 통과했는가 (npm test)
2. console.log 등 디버그 코드가 남아있지 않은가
3. 변경사항 diff를 처음부터 끝까지 읽었는가
4. 커밋 메시지/PR 제목이 변경 내용을 명확히 설명하는가

이 항목을 하나씩 확인하고, 결과를 요약해서 PR 설명을 작성해줘.
```

이렇게 한 번만 만들어두면, 다음부터는 채팅창에 /pr-checklist라고 한 줄만 치면 됩니다.

Claude Code가 이 파일을 읽어서 그 안에 적힌 절차(테스트 실행 확인, diff 검토, PR 설명 작성)를 그대로 수행합니다. 즉 "매번 다시 설명하기" → "한 번 가르쳐 두고 이름만 부르기"로 바뀌게 됩니다.

"Skill이 왜 필요한가" 와 닿나요?

CLAUDE.md는 "이 프로젝트는 이런 규칙이야"를 매 세션 자동으로 알려주는 것이고, Skill은 "이 작업을 시킬 때는 이 절차대로 해줘"를 필요할 때만 불러 쓰는 것이 차이입니다.

"여러분이 팀플을 할 때, 이런 체크리스트를 팀 전체가 똑같이 지키게 만드는 도구"라 볼 수도 있습니다.

Hook이란?

- 특정 이벤트(파일 저장 직후, 커밋 직전 등)에 무조건 자동으로 실행되는 규칙
- Skill은 "Claude가 알아서 판단해서 쓰는 것", Hook은 "무조건 실행되는 규칙"이라는 차이
- 예: 코드 수정 시 자동 포맷터 실행, 커밋 전 테스트 자동 실행

시간 관계상 오늘은 개념만 짚고 넘어갑니다.

PART 3

프로젝트 컨텍스트 관리: CLAUDE.md와 AGENTS.md

(AGENTS.md 선택)

- [실습] CLAUDE.md 작성 + /context로 확인
- AGENTS.md란 무엇인가 (선택)

CLAUDE.md 작성하기

```
# 프로젝트 개요
바닐라 HTML/CSS/JS 할 일 관리 웹페이지. 새 기능 추가 실습용.

# 파일 구조
- todoLogic.js: 할 일 목록을 다루는 순수 함수들
- app.js: 화면(DOM)과 todoLogic.js를 연결하는 코드

# 코딩 컨벤션
- 함수/변수명은 camelCase
- 새 기능 추가 시 tests/ 에 Jest 테스트 추가
```

직접 타이핑하거나, `"/init 실행해줘"`로 자동 초안을 받아 다듬어도 됩니다. 작성 후 `/context`로 로드됐는지 꼭 확인하세요.

AGENTS.md란 무엇인가

- Cursor, Codex 등 여러 AI 코딩 도구가 공통으로 읽을 수 있게 만든 업계 표준 컨텍스트 파일
- Claude Code는 기본적으로 AGENTS.md를 직접 읽지 않고, CLAUDE.md만 읽습니다.
- 이미 AGENTS.md가 있다면, CLAUDE.md 맨 위에 한 줄만 적어 내용을 불러올 수 있습니다:

```
@AGENTS.md
```

```
## Claude Code 전용 지침  
(Claude Code에서만 필요한 내용)
```

명세로 시작하기:

PRD·TRD·REQUIREMENTS.md와 SDD

28분

- 명세를 사용하는 이유 & SDD 개념
- PRD vs TRD vs REQUIREMENTS.md 비교
- [실습] 미니 기능 REQUIREMENTS.md 작성

명세를 쓰는 이유 & SDD

- 재현성 — 말로 대충 시키면 같은 요청도 매번 다르게 해석됨
- 커뮤니케이션 비용 감소 — 매번 처음부터 설명하지 않아도 됨
- 리뷰 효율 — "명세대로 됐는지"만 확인하면 됨
- 요구사항 누락 방지 — 글로 적으면 빠진 조건이 드러남
- 스코프 크립 방지 — "오늘 할 일"의 경계가 문서로 못박힘

SDD (Spec-Driven Development)

명세 작성 → agent에게 명세를 주고 구현 요청 → 결과를 명세와 대조해 검증 → 어긋나면 명세/구현 중 무엇을 고칠지 판단

PRD vs TRD vs REQUIREMENTS.md

문서	초점	담는 내용
PRD	무엇을, 왜	대상 사용자, 문제, 성공 기준
TRD	어떻게	아키텍처, 입출력 형식, 제약조건
REQUIREMENTS.md	소규모 실전용	PRD+TRD를 가볍게 합친 문서 (오늘 사용)

팀이 크고 "왜"와 "어떻게"를 다른 사람이 결정한다면 PRD/TRD 분리, 소규모면 REQUIREMENTS.md 하나로 충분합니다.

REQUIREMENTS.md 예시

기능명: 마감일 + 우선순위 태그

목적

할 일에 마감과 중요도를 표시해 우선순위를 파악하기 위해

기능 목록

- 할 일 추가 시 마감일 (YYYY-MM-DD) 과 우선순위 (상/중/하) 선택 입력
- 목록 표시 시 마감일이 빠른 순으로 정렬 (옵션)

입출력 예시

```
addTodo("보고서 작성", { due: "2026-09-10", priority: "high" })
```

제약

- 기존 데이터 구조의 하위 호환을 깨면 안 됨

미니 기능 REQUIREMENTS.md 작성

- 배정받은 신규 기능 중 하나를 골라 앞의 템플릿을 채워서 작성합니다
 - ① 마감일 + 우선순위 태그
 - ② 완료 항목 필터링/정렬
 - ③ 검색(키워드 필터링)
- 작성 후 짝과 파일을 교환해 "이 문서만 보고 구현할 수 있을까?" 상호 점검

이 REQUIREMENTS.md는 6부(종합 실습)에서 그대로 재사용됩니다 — 잘 챙겨두세요.

PART 5

AI와 협업하는 개발 방법론

32분

- [실습] 작은 단위로 쪼개기 & 명확한 지시
- 리뷰: 결과를 맹신하지 않기
- 증강코딩 vs 바이트코딩
- TDD: Red → Green → Refactor, SDD와 비교

작은 단위로 쪼개기 & 명확한 지시

나쁜 지시

"완료된 거 지우는 기능도 넣어줘"

좋은 지시

"todoLogic.js에 완료(done=true) 항목만 삭제하는 clearDone(todos) 함수를 추가해줘. 삭제된 개수를 함께 반환해줘. 기존 removeTodo는 건드리지 마. 테스트도 추가해줘."

좋은 지시의 4요소

목표(무엇을) · 입출력 예시(구체적 형태) · 제약(건드리면 안 되는 것) · 완료 기준(어떻게 확인)

리뷰: 결과를 망신하지 않기

- GitHub Desktop의 Changes 탭으로 돌아가 실제 어떤 파일이 왜 바뀌었는지 훑어보기
- 의도치 않은 파일까지 건드리진 않았는지 확인
- agent에게 "왜 이렇게 짰는지" 되물어 근거를 확인하는 습관
- 마음에 안 들면 되돌리기 — 커밋 전이면 Discard changes, VS Code 확장의 체크포인트(되감기)로도 복구 가능

"일단 시켜보고 마음에 안 들면 되돌리면 된다"는 안전망이 오히려 더 과감한 실습을 가능하게 합니다.

증강코딩 vs 바이브코딩

전부 손으로 타이핑

증강코딩 (Augmented Coding)

바이브코딩 (결과를 보지 않고 계속 지시)

- 증강코딩: 설계·검증의 주도권은 사람이 쥐고, AI는 속도를 높이는 역할
- 바이브코딩이
 - 괜찮은 경우: 프로토타입, 일회성 스크립트
 - 위험한 경우: 프로덕션, 보안·결제·개인정보 관련 로직

오늘 6부 실습은 의도적으로 "증강코딩" 방식(명세+테스트로 방향을 잡고 AI가 구현)으로 설계되었습니다.

TDD: Red → Green → Refactor

Red

테스트를 실행하면 당연히 실패한다

이 실패가 곧 우리가 만들 기능의 명세서 역할

Green

테스트를 통과시킬 만큼만 최소로 구현

예쁘게 짜는 건 다음 단계

Refactor

테스트가 통과하는 상태를 유지하며 코드 정리

테스트가 "안전망" 역할

이 세 단계를 계속 반복하며 기능을 완성해나갑니다.

테스트는 Jest(자바스크립트용)로 작성합니다. 문법은 몰라도 됩니다 — Claude Code가 작성부터 npm install, 실행까지 다 하고, 여러분은 권한 요청에서 Allow만 누르면 됩니다.

SDD vs TDD

구분	SDD (명세 주도)	TDD (테스트 주도)
먼저 쓰는 것	요구사항 문서(REQUIREMENTS.md)	테스트 코드(Jest)
답하는 질문	무엇을/왜 만드는가	코드가 올바르게 동작하는가
검증 방법	사람이 명세와 결과물 대조	테스트 실행 결과(자동)
AI 협업 역할	"무엇을/왜"를 알려주는 나침반	"됐는지"를 알려주는 자동 채점기

SDD가 "방향"을, TDD가 "완료 판정"을 줍니다 — 결합하면 사람이 매번 눈으로 확인하지 않아도 되는 사이클이 만들어집니다.

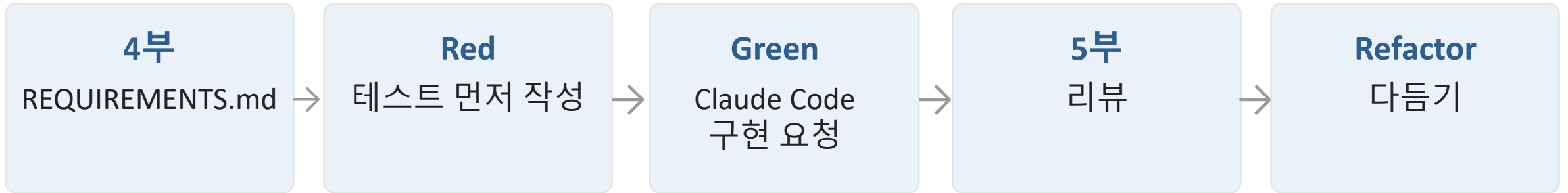
PART 6

종합 실습: SDD + TDD 결합 미니 프로젝트

45분

- 실습 시나리오 설명
- [실습] 1단계 — 테스트 먼저 작성 (Red)
- [실습] 2단계 — 구현 요청 (Green)
- [실습] 3단계 — 리뷰 및 커밋 (Refactor)
- 결과 공유 및 디브리핑

오늘의 전체 사이클



- 짝과 함께, 자기 REQUIREMENTS.md 기준으로 진행, 막히면 손 들기(손들기 전 짝과 먼저 상의)
- 시간 안내: 1단계 10분 / 2단계 15분 / 3단계 10분

테스트 케이스 먼저 작성

- 자기 REQUIREMENTS.md의 "기능 목록"을 보고 Claude Code에게 Jest 테스트 작성을 요청

@REQUIREMENTS.md 를 참고해서, tests/todoLogic.test.js에
이 기능의 정상 케이스 2개와 에러 케이스 1개에 대한
Jest 테스트를 작성해줘. 아직 구현은 하지 마.

테스트를 실행하면 당연히 실패(Red)합니다 — 이 빨간불이 다음 단계의 목표를 알려주는 지도입니다.

Claude Code에게 구현 요청

- REQUIREMENTS.md와 방금 작성한 테스트 파일을 컨텍스트로 주고 구현 요청

@REQUIREMENTS.md 와 @tests/todoLogic.test.js 를 참고해서,
이 테스트가 모두 통과하도록 todoLogic.js에 기능을 구현해줘.
CLAUDE.md의 컨벤션을 지켜줘.
구현 후 npm test를 실행해서 결과를 보여줘.

테스트가 통과(Green)할 때까지 반복 — 실패하면 어떤 테스트가 왜 실패했는지 물어보고 재지시하세요.

리뷰 및 마무리

- GitHub Desktop의 Changes 탭으로 변경사항을 처음부터 끝까지 훑어보기
- 필요하면 "테스트를 유지한 채로 이 함수 조금 더 깔끔하게 정리해줘"라고 Refactor 요청
- "커밋 메시지를 만들어줘"라고 요청 → GitHub Desktop 커밋창에 붙여넣어 Commit to main
- 시간 남으면 REQUIREMENTS.md의 나머지 조건(에러 케이스 등)까지 추가 반복

결과 공유 및 디브리핑

- 옆 사람과 1분씩: "어디서 SDD(명세)가, 어디서 TDD(Red-Green-Refactor)가 도움이 됐는지" 공유
- 전체 공유: 흥미로운 시행착오 사례 1~2개 발표

PART 7

마무리

10분

- 오늘의 체크리스트 정리
- 자주 하는 실수 & Q&A
- 다음 시간 예고

오늘의 체크리스트

- 환경

- GitHub Desktop(Clone·Changes·Commit) · Claude Code for VS Code 설치

- 도구

- 슬래시 커맨드(/clear /compact /model /context /help) · CLAUDE.md · Skill·Hook

- 명세

- PRD/TRD/REQUIREMENTS.md, 명세를 쓰는 이유

- 협업 방법론

- 쪼개기·명확한 지시 · 리뷰 · 증강코딩 · SDD+TDD(Red-Green-Refactor)

자주 하는 실수 & Q&A

- 명세 없이 바로 시키기 → 재현 안 되는 결과, 스코프 크립
- Changes 탭을 안 열어보고 그냥 커밋하기 → 의도치 않은 부작용을 나중에 발견
- 테스트 없이 "됐다"는 말만 믿고 넘어가기 → 회귀 버그

회귀 버그(regression bug): 예전에는 잘 동작하던 기능이, 다른 부분을 수정하는 과정에서 다시 고장 나는 버그

Q&A

오늘 만든 코드는 GitHub Desktop으로 커밋해서 가져가세요.

다음 시간 예고



Matt Pocock

@mattpocockuk · 구독자 37.3만명 · 동영상 299개

Become an AI Hero with tips, tricks and tutorials. We don't do vibe coding - this is a chan...더보기

aihero.dev 외 링크 1개

구독

<https://www.youtube.com/@mattpocockuk>